Home lab project week 3 day 3

Task
Configure and test the virtual network between Kali Linux and Ubuntu using controlled connectivity tests.

In my lab setup, there are :

- **MacBook** → physical host
- **UTM** → virtualization platform
- **Kali Linux** → testing/analyst system — ip `192.168.64.2`
- **Ubuntu** → protected system — ip `192.168.64.3`
- **Virtual network** → network ip `192.168.64.0/24`

> This network is set up to be network **isolated from your physical/home network**

Activities:

1. Verify the virtual network configuration

On both VMs, I need to verify that the network adapter is connected to the same isolated/host-only virtual network.

Kali:

ip addr

You should see an address similar to:

192.168.64.2/24

```
┌──(kali㉿kali)-[~]
└─$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether ea:f0:2f:00:11:3a brd ff:ff:ff:ff:ff:ff
    inet 192.168.64.2/24 brd 192.168.64.255 scope global dynamic noprefixrout
e eth0
       valid_lft 2222sec preferred_lft 2222sec
    inet6 fda6:6bc3:cab9:adbc:1403:4cbc:c0a3:bcc6/64 scope global temporary d
ynamic
       valid_lft 529624sec preferred_lft 10973sec
    inet6 fda6:6bc3:cab9:adbc:e8f0:2fff:fe00:113a/64 scope global dynamic mng
tmpaddr noprefixroute
       valid_lft 2591978sec preferred_lft 604778sec
    inet6 fe80::e8f0:2fff:fe00:113a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Ubuntu:

When we type the common=and: ip addr

We see an address of 192.168.64.3/24

```
root@ubuntu:/home/ubuntu# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 1a:63:23:39:a4:2c brd ff:ff:ff:ff:ff:ff
    inet 192.168.64.3/24 metric 100 brd 192.168.64.255 scope global dynamic enp0
s1
       valid_lft 3429sec preferred_lft 3429sec
    inet6 fda6:6bc3:cab9:adbc:1863:23ff:fe39:a42c/64 scope global dynamic mngtmp
addr noprefixroute
       valid_lft 2591976sec preferred_lft 604776sec
    inet6 fe80::1863:23ff:fe39:a42c/64 scope link
       valid_lft forever preferred_lft forever
root@ubuntu:/home/ubuntu#
```

As we can see both systems are on the same network: 192.168.64.0/24

---

2. Identify the network interfaces

On Kali:

or:

ip addr

Identify the interface connected to the lab network. It is: `eth0`

On Ubuntu:

or:

ip addr

Identify its corresponding interface. It is : `enp0s1`

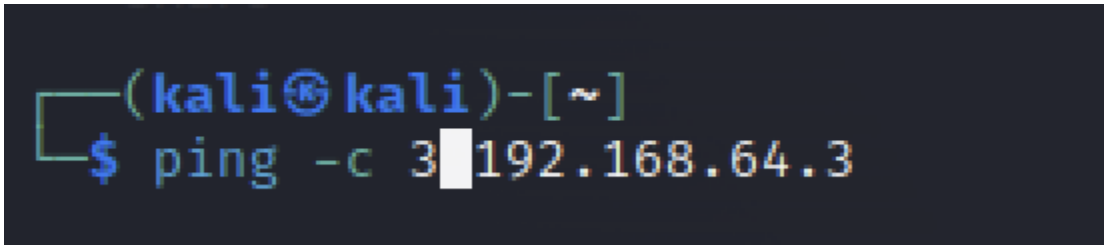
This gives us documentation showing **which interface is being used for the lab network**.

---

3. Test Kali → Ubuntu connectivity

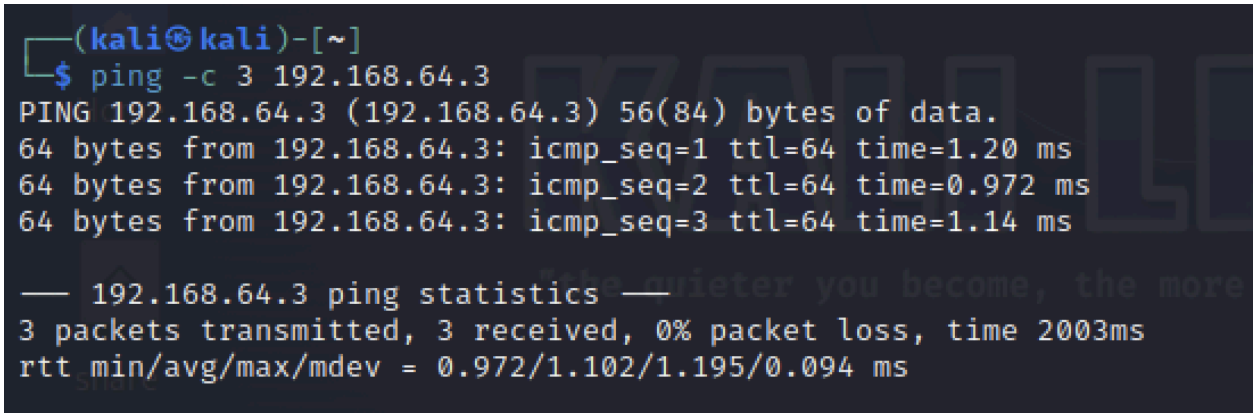
From Kali, ping Ubuntu:

ping -c 4 192.168.64.3



A successful result should look similar to:

64 bytes from 192.168.64.3: icmp_seq=1 ttl=64 time=...
64 bytes from 192.168.64.3: icmp_seq=2 ttl=64 time=...



This demonstrates that Kali can communicate with Ubuntu.

Record:

- Destination IP 192.168.64.3
- Number of packets sent : 3 packets
- Number received: 3 packets
- Packet loss: 0 packets
- Approximate response time:2003ms

---

4. Test Ubuntu → Kali connectivity

From Ubuntu:

ping -c 4 192.168.64.2

This tests the connection in the opposite direction.

We want to demonstrate that communication works between the two systems, rather than only testing one direction.

```
root@ubuntu:/home/ubuntu# ping -c 3 192.168.64.2
PING 192.168.64.2 (192.168.64.2) 56(84) bytes of data.
64 bytes from 192.168.64.2: icmp_seq=1 ttl=64 time=0.816 ms
64 bytes from 192.168.64.2: icmp_seq=2 ttl=64 time=1.11 ms
64 bytes from 192.168.64.2: icmp_seq=3 ttl=64 time=1.46 ms

--- 192.168.64.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 0.816/1.125/1.456/0.261 ms
root@ubuntu:/home/ubuntu#
```

---

5. Verify the routing configuration

On Kali:

ip route

```
┌──(kali㉿kali)-[~]
└─$ ip route
default via 192.168.64.1 dev eth0 proto dhcp src 192.168.64.2 metric 100
192.168.64.0/24 dev eth0 proto kernel scope link src 192.168.64.2 metric 100
```

On Ubuntu:

ip route

we should see a route associated with the `192.168.64.0/24` network.

192.168.64.0 dev eth0 proto kernel scope link src 192.168.64.2 metric 100

- **`default via 192.168.64.1`**: Any traffic destined for an external network (like the internet) is sent to the router/gateway at IP address `192.168.64.1`.

- **`dev eth0`**: This traffic leaves the machine via the first wired network interface, named `eth0`.
- **`proto dhcp`**: The route was automatically configured by a DHCP server on the network.
- **`src 192.168.64.2`**: The local IP address assigned to this interface is `192.168.64.2`.
- **`metric 100`**: The routing priority value. Lower metric values are preferred if there are multiple routes to the same destination.

```
root@ubuntu:/home/ubuntu# ip route
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.3 metric 100
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.3 metric 100
192.168.64.1 dev enp0s1 proto dhcp scope link src 192.168.64.3 metric 100
root@ubuntu:/home/ubuntu#
```

Default via 192.168.64.1 dev enps1 proto dhcp src 192.168.64.3 metric 100
192.168.64.0/24 dev enp0s1 prot kernel scope link src 192.168.64.3 metric 100

This confirms that each VM knows how to reach the other VM through the virtual network.

---

6. Perform a basic controlled connectivity test

Let's first try to find out if nmap is installed in kali

```
┌──(kali㉿kali)-[~]
└─$ nmap --version
Nmap version 7.94 ( https://nmap.org )
Platform: aarch64-unknown-linux-gnu
Compiled with: liblua-5.4.4 openssl-3.0.10 libssh2-1.10.0 libz-1.2.13 libpcre-8
.39 libpcap-1.10.4 nmap-libdnet-1.12 ipv6
Compiled without:
Available nsock engines: epoll poll select
```

From Kali, the command used is:

nmap -sn 192.168.64.0/24

This is a **host discovery scan** of my isolated lab network.

We should be able to identify Ubuntu system at:

192.168.64.3

---

7. Test a specific service

If Ubuntu has SSH enabled, for example:

nmap -p 22 192.168.64.3
The return will be:
PORT STATE SERVICE
22/tcp open ssh

The important point is that the test is controlled and performed against my own lab system.

---

8. Confirm network isolation

This is an important part of this home lab.

The objective is not simply: "Kali can ping Ubuntu."

but , I also want to demonstrate that:

> "Kali and Ubuntu can communicate with each other on the isolated lab network without unnecessarily exposing the vulnerable/protected systems to my normal MacBook/home network."

Evidence to collect for Day 3

I would take **5–6 screenshots** for your project documentation:

Evidence	Screenshot
1	Kali `ip addr` showing `192.168.64.2`

2	Ubuntu `ip addr` showing `192.168.64.3`
3	Kali pinging Ubuntu successfully
4	Ubuntu pinging Kali successfully
5	`ip route` on Kali/Ubuntu
6	Kali `nmap -sn 192.168.64.0/24` showing the lab hosts

These screenshots are evidence that I **configured and tested the network**, rather than simply stating that it works.

What Day 3 accomplishes

By the end of Day 3, I was able to accomplish the following:

Virtual Network → IP Configuration → Connectivity → Routing → Host Discovery → Controlled Testing

My Day 3 result could therefore be summarized as:

> **Kali Linux (`192.168.64.2`) and Ubuntu (`192.168.64.3`) were configured on the isolated `192.168.64.0/24` virtual network. Controlled ping, routing, and host-discovery tests were performed to verify communication between the testing and protected systems.**